

МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение высшего
образования

НИЖЕГОРОДСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ

УНИВЕРСИТЕТ им. Р.Е.АЛЕКСЕЕВА



Институт радиоэлектроники и информационных технологий

Кафедра “Вычислительные системы и технологии”

WEB - технологии

Отчет

по лабораторной работе №3

Выполнил студенты группы 23-ПО-2

Блошкин Алексей Юрьевич

_____ «__» _____ 2025 г.

Проверил преподаватель кафедры “ВСТ”

Скорынин С. С

_____ «__» _____ 2025г.

Г. Нижний Новгород, 2025

1. ОПИСАНИЕ ПРОЕКТА

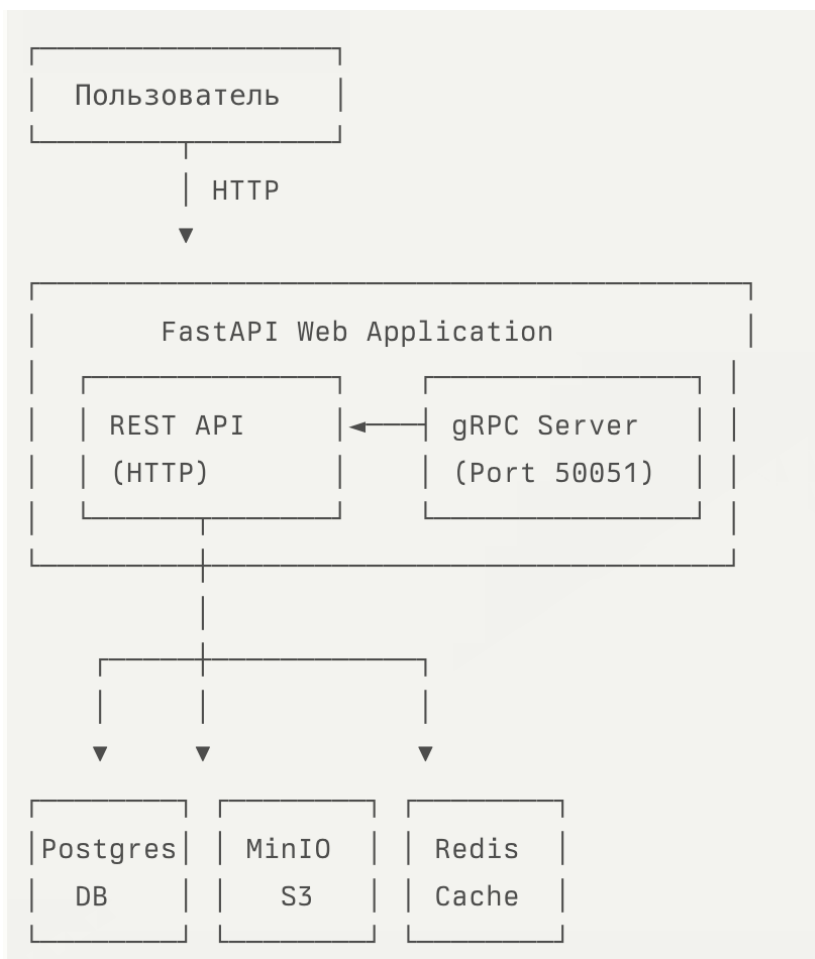
"Новостник 2025" - это полнофункциональное веб-приложение для управления новостным контентом с ролевой моделью доступа, реализованное на современном технологическом стеке.

Ключевые особенности:

- REST API на базе FastAPI для всех операций с данными
- gRPC сервис для высокопроизводительных межсервисных коммуникаций
- PostgreSQL для надёжного хранения данных
- MinIO S3 для хранения изображений
- Redis для кэширования
- JWT аутентификация с ролевой моделью
- Docker Compose для оркестрации всех сервисов

2. АРХИТЕКТУРА СИСТЕМЫ

2.1 Общая схема



2.2 Микросервисная архитектура

Проект разделён на 4 основных сервиса:

1. news-app (FastAPI) - основное приложение
2. news-postgres - база данных PostgreSQL
3. news-minio - объектное хранилище S3
4. news-redis - система кэширования

3. РЕАЛИЗАЦИЯ ТРЕБОВАНИЙ ЗАДАНИЯ

ЗАДАНИЕ 1: Frontend приложение

Статус: Частично выполнено

Реализация:

- Создан веб-интерфейс на базе HTML/CSS/JavaScript
- Использован Tailwind CSS для современного дизайна
- Реализована динамическая отрисовка через JavaScript
- REST API взаимодействие через fetch() API
- JWT токены сохраняются в localStorage и cookies

Отличие от требований:

Вместо отдельного SPA приложения (React/Angular/Vue) использован подход Server-Side Rendering + Client-Side JavaScript с шаблонами Jinja2.

Функциональность Frontend:

1. Авторизация/регистрация пользователей
2. Просмотр списка новостей
3. Детальный просмотр новости
4. Создание/редактирование/удаление новостей (для авторов/админов)
5. Профиль пользователя с историей чтения
6. Поиск по новостям
7. Адаптивный дизайн

ЗАДАНИЕ 2.1: База данных PostgreSQL

Статус: Полностью выполнено

Проектирование БД

Таблица users:

sql

CREATE TABLE users (

```
id SERIAL PRIMARY KEY,  
username VARCHAR UNIQUE NOT NULL,  
email VARCHAR UNIQUE NOT NULL,  
hashed_password VARCHAR NOT NULL,  
role VARCHAR DEFAULT 'reader',  
is_active BOOLEAN DEFAULT TRUE,  
created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
read_history JSON DEFAULT '[]'  
);
```

Таблица news:

sql

```
CREATE TABLE news (  
    id SERIAL PRIMARY KEY,  
    title VARCHAR(200) NOT NULL,  
    author VARCHAR(100) NOT NULL,  
    content TEXT NOT NULL,  
    summary VARCHAR(500) NOT NULL,  
    image_url VARCHAR,  
    tags VARCHAR[] DEFAULT '{}',  
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,  
    updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

ORM - SQLAlchemy 2.0

Использован SQLAlchemy 2.0 (эквивалент Entity Framework для Python):

python

```
# app/models/user.py
```

```
from sqlalchemy import Column, Integer, String, Boolean, DateTime, JSON  
from datetime import datetime
```



```
from app.db.base import Base
```

```
class User(Base):
```

```
    __tablename__ = "users"
```

```
    id = Column(Integer, primary_key=True, index=True)
```

```
    username = Column(String, unique=True, index=True, nullable=False)
```

```
    email = Column(String, unique=True, index=True, nullable=False)
```

```
    hashed_password = Column(String, nullable=False)
```

```
    role = Column(String, default="reader")
```

```
    is_active = Column(Boolean, default=True)
```

```
    created_at = Column(DateTime, default=datetime.utcnow)
```

```
    read_history = Column(JSON, default=list)
```

Code-First подход

Все модели определены в коде Python, миграции управляются через Alembic (аналог EF Migrations):

```
python
```

```
# Создание таблиц при старте
```

```
Base.metadata.create_all(bind=engine)
```

Dependency Injection

Реализован DI паттерн через FastAPI:

```
python
```

```
from app.db.session import get_db
```

```
@router.get("/")
```

```
async def get_news(db: Session = Depends(get_db)):
```

```
    return news_service.get_all(db)
```

Repository Pattern

CRUD операции вынесены в отдельный слой:

```
app/
├── crud/      # Repository слой
|   └── news.py  # CRUD для новостей
├── services/   # Бизнес-логика
|   └── news_service.py
└── api/       # API endpoints
    └── routes.py
```

Пример репозитория:

```
# app/crud/news.py
```

```
def create(db: Session, news_in: NewsCreate) -> News:
```

```
    db_news = News(**news_in.dict())
```

```
    db.add(db_news)
```

```
    db.commit()
```

```
    db.refresh(db_news)
```

```
    return db_news
```

```
def get_all(db: Session):
```

```
    return db.query(News).all()
```

Тестовые данные

Автоматическая загрузка начальных данных при старте:

```
# app/init_data.py
```

```
def load_initial_data():
```

```
    # 3 тестовых пользователя
```

```
    users = [admin, author, reader]
```

```
    # 3 тестовые новости
```

```
    news = [news1, news2, news3]
```

ЗАДАНИЕ 2.2: Docker контейнеризация

Статус: Полностью выполнено

Dockerfile

FROM python:3.11-slim

RUN apt-get update && apt-get install -y \
gcc libpq-dev && rm -rf /var/lib/apt/lists/*

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

RUN useradd -m appuser

USER appuser

EXPOSE 8000 50051

CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

ЗАДАНИЕ 2.3: Docker Compose

Статус: Полностью выполнено

docker-compose.yml

version: '3.8'

services:

PostgreSQL база данных

postgres:

image: postgres:15-alpine

container_name: news-postgres

environment:

POSTGRES_USER: newsuser

POSTGRES_PASSWORD: newspass

POSTGRES_DB: newsdb

volumes:

- postgres_data:/var/lib/postgresql/data

ports:

- "5432:5432"

MinIO S3 хранилище

minio:

image: minio/minio:latest

container_name: news-minio

command: server /data --console-address ":9001"

environment:

MINIO_ROOT_USER: minioadmin

MINIO_ROOT_PASSWORD: minioadmin

volumes:

- minio_data:/data

ports:

- "9000:9000"

- "9001:9001"

Redis кэш

redis:

image: redis:alpine

container_name: news-redis

ports:

- "6379:6379"

Основное приложение

app:

build: .

container_name: news-app

depends_on:

- postgres

- minio

- redis

environment:

DATABASE_URL: postgresql://newsuser:newspass@postgres:5432/newsdb

MINIO_ENDPOINT: minio:9000

REDIS_URL: redis://redis:6379

ports:

- "8000:8000"

- "50051:50051"

volumes:

postgres_data:

minio_data:

Команды для запуска:

docker compose up --build # Сборка и запуск

docker compose down -v # Остановка и очистка

docker compose logs app # Просмотр логов

ЗАДАНИЕ 2.4: Аутентификация API

Статус: Полностью выполнено

JWT аутентификация

Реализована полноценная JWT аутентификация с использованием библиотеки python-jose:

```
# app/core/security.py
```

```
from jose import jwt
```

```
from passlib.context import CryptContext
```

```
SECRET_KEY = "super-secret-jwt-key-change-in-prod-2025"
```

```
ALGORITHM = "HS256"
```

```
ACCESS_TOKEN_EXPIRE_MINUTES = 1440 # 24 часа
```

```
def create_access_token(data: dict):
```

```
    to_encode = data.copy()
```

```
    expire = datetime.utcnow() +  
timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
```

```
    to_encode.update({"exp": expire})
```

```
    return jwt.encode(to_encode, SECRET_KEY, algorithm=ALGORITHM)
```

Endpoints для аутентификации

```
# POST /auth/register - регистрация
```

```
# POST /auth/login - авторизация
```

```
# GET /profile - профиль (требуется токен)
```

Пример использования:

```
# Логин
```

```
curl -X POST http://localhost:8000/auth/login \
```

```
-d "username=admin&password=admin"
```

Ответ

```
{  
  "access_token": "eyJhbGciOiJIUzI1NiIs...",  
  "token_type": "bearer",  
  "username": "admin",  
  "role": "admin"  
}
```

Защита endpoints

```
from app.core.security import get_current_user
```

```
@router.post("/news")  
async def create_news(  
    news_in: NewsCreate,  
    current_user: AuthenticatedUser = Depends(require_admin)  
):  
    return await news_service.create(db, news_in)
```

ЗАДАНИЕ 3.1: Ролевая модель авторизации

Статус: Полностью выполнено

Роли пользователей

Реализованы 4 роли с разными уровнями доступа:

Роль	Доступ
anonymous	Чтение новостей
reader	Чтение + регистрация + профиль + история
author	Всё выше + создание/редактирование своих статей
admin	Полный доступ ко всему контенту

Декораторы авторизации

```
# app/core/security.py
```

```
def require_reader(user: AuthenticatedUser = Depends(get_current_user)):
    if user.role not in ["reader", "author", "admin"]:
        raise HTTPException(status_code=403)
    return user
```

```
def require_author(user: AuthenticatedUser = Depends(get_current_user)):
    if user.role not in ["author", "admin"]:
        raise HTTPException(status_code=403)
    return user
```

```
def require_admin(user: AuthenticatedUser = Depends(get_current_user)):
    if user.role != "admin":
        raise HTTPException(status_code=403)
    return user
```

Применение в API

```
# Публичный доступ
```

```
@router.get("/news")
```

```
async def get_news(
```

```
    current_user: Union[AuthenticatedUser, AnonymousUser] =
    Depends(get_current_user_optional)
```

```
):
```

```
    return await news_service.get_all(db)
```

```
# Только для авторов и админов
```

```
@router.post("/news")
```

```
async def create_news(
```



```

news_in: NewsCreate,
current_user: AuthenticatedUser = Depends(require_author)
):
    return await news_service.create(db, news_in)

# Только для админов
@router.delete("/news/{id}")
async def delete_news(
    id: int,
    current_user: AuthenticatedUser = Depends(require_admin)
):
    return await news_service.delete(db, id)

```

ЗАДАНИЕ 3.2: gRPC сервис

Статус: Полностью выполнено

Определение proto файла

// app/protos/news.proto

```
syntax = "proto3";
```

```
package news;
```

```

service NewsService {
    rpc GetAllNews (Empty) returns (NewsListResponse);
    rpc GetNewsById (NewsIdRequest) returns (NewsResponse);
    rpc CreateNews (CreateNewsRequest) returns (NewsResponse);
    rpc UpdateNews (UpdateNewsRequest) returns (NewsResponse);
}

```

```
rpc DeleteNews (NewsIdRequest) returns (DeleteResponse);  
rpc SearchNews (SearchRequest) returns (NewsListResponse);  
}
```

```
message NewsResponse {  
    int32 id = 1;  
    string title = 2;  
    string author = 3;  
    string content = 4;  
    string summary = 5;  
    string image_url = 6;  
    repeated string tags = 7;  
    string created_at = 8;  
}
```

gRPC сервер

```
# app/grpc/server.py  
  
import grpc  
  
from concurrent import futures  
  
from app.protos import news_pb2_grpc  
from app.grpc.news_service import NewsServicer  
  
def serve_grpc():  
    server = grpc.server(futures.ThreadPoolExecutor(max_workers=10))  
    news_pb2_grpc.add_NewsServiceServicer_to_server(NewsServicer(), server)  
    server.add_insecure_port('[::]:50051')  
    server.start()  
    server.wait_for_termination()
```

Запуск в отдельном потоке

```
# app/main.py

import threading

@app.on_event("startup")
async def on_startup():
    grpc_thread = threading.Thread(target=serve_grpc, daemon=True)
    grpc_thread.start()
    logger.info("gRPC-сервер запущен на порту 50051")
```

Тестирование gRPC

```
# Через grpcurl
grpcurl -plaintext localhost:50051 news.NewsService/GetAllNews
```

Результат: Сервис одновременно работает по REST (HTTP/8000) и gRPC (50051).

ЗАДАНИЕ 3.3: S3 хранилище (MinIO)

Статус: Полностью выполнено

Конфигурация MinIO

```
# app/utils/s3.py

import boto3

from app.core.config import settings

s3_client = boto3.client(
    's3',
    endpoint_url=f'http://{settings.S3_ENDPOINT}',
    aws_access_key_id=settings.S3_ACCESS_KEY,
    aws_secret_access_key=settings.S3_SECRET_KEY
)
```

Загрузка изображений

```
@router.post("/upload")
```

```
async def upload_image(file: UploadFile):
```

```
    filename = f"{uuid.uuid4()}_{file.filename}"
```

```
    # Загрузка в S3
```

```
    s3_client.upload_fileobj(
```

```
        file.file,
```

```
        settings.S3_BUCKET,
```

```
        filename
```

```
    )
```

```
    # URL изображения
```

```
    image_url = f"http://{settings.S3_ENDPOINT}/{settings.S3_BUCKET}/{filename}"
```

```
    return {"url": image_url}
```

Использование в новостях

```
# При создании новости
```

```
if image_file:
```

```
    # Загружаем в S3
```

```
    image_url = upload_to_s3(image_file)
```

```
    news_in.image_url = image_url
```

Преимущества:

- Разгрузка основной БД
- Масштабируемость хранения
- CDN совместимость
- Независимое управление медиа

ЗАДАНИЕ 3.4: Redis кэширование

Статус: Полностью выполнено

Подключение Redis

```
# app/core/cache.py
```

```
import redis
```

```
from app.core.config import settings
```

```
redis_client = redis.from_url(settings.REDIS_URL, decode_responses=True)
```

```
def get_cached(key: str):
```

```
    return redis_client.get(key)
```

```
def set_cache(key: str, value: str, expire: int = 300):
```

```
    redis_client.setex(key, expire, value)
```

Кэширование списка новостей

```
@router.get("/news")
```

```
async def get_news(db: Session = Depends(get_db)):
```

```
    # Проверяем кэш
```

```
    cached = get_cached("news:all")
```

```
    if cached:
```

```
        return json.loads(cached)
```

```
    # Запрос к БД
```

```
    news = await news_service.get_all(db)
```

```
    # Сохраняем в кэш на 5 минут
```

```
    set_cache("news:all", json.dumps(news), 300)
```

```
    return news
```

Инвалидация кэша

```
@router.post("/news")

async def create_news(news_in: NewsCreate):

    # Создаём новость

    news = await news_service.create(db, news_in)


    # Инвалидируем кэш

    redis_client.delete("news:all")

    return news
```

ЗАДАНИЕ 3.5: Docker Compose для всех сервисов

Статус: Полностью выполнено

Все сервисы добавлены в docker-compose.yml:

services:

```
app:      # FastAPI приложение
postgres: # База данных
minio:    # S3 хранилище
redis:    # Кэш
```

Одна команда запускает всю инфраструктуру:

```
docker compose up
```

4. ТЕХНОЛОГИЧЕСКИЙ СТЕК

Backend

- FastAPI 0.104+ - современный веб-фреймворк
- Python 3.11 - язык программирования
- SQLAlchemy 2.0 - ORM для работы с БД
- Alembic - система миграций
- Pydantic 2.0 - валидация данных
- python-jose - JWT токены
- passlib + bcrypt - хеширование паролей
- grpcio - gRPC сервер

- boto3 - клиент для S3
- redis-py - клиент для Redis

База данных

- PostgreSQL 15 - реляционная БД
- pgAdmin (опционально) - администрирование

Хранилище

- MinIO - S3-совместимое хранилище

Кэширование

- Redis 7 - in-memory кэш

Frontend

- HTML5 + CSS3
- JavaScript (ES6+)
- Tailwind CSS - UI фреймворк
- Jinja2 - шаблонизатор

DevOps

- Docker 24+
- Docker Compose
- GitHub - система контроля версий

5. СТРУКТУРА ПРОЕКТА

LABA3/

```

└─ app/
|   └─ api/          # REST API endpoints
|   |   └─ auth.py   # Аутентификация
|   |   └─ news.py   # CRUD для новостей
|   |   └─ routes.py  # Основные маршруты
|   └─ core/         # Ядро приложения
|   |   └─ config.py  # Настройки
|   |   └─ security.py # JWT + авторизация

```

- | | └─ cache.py # Redis кэш
- | └─ crud/ # Repository паттерн
- | | └─ news.py # CRUD операции
- | └─ db/ # База данных
- | | └─ base.py # Base модель
- | | └─ session.py # Сессии БД
- | └─ grpc/ # gRPC сервис
- | | └─ server.py # gRPC сервер
- | | └─ news_service.py
- | └─ models/ # ORM модели
- | | └─ user.py # Модель User
- | | └─ news.py # Модель News
- | └─ schemas/ # Pydantic схемы
- | | └─ user.py # DTO для User
- | | └─ news.py # DTO для News
- | └─ services/ # Бизнес-логика
- | | └─ news_service.py
- | └─ utils/ # Утилиты
- | | └─ s3.py # S3 клиент
- | └─ protos/ # gRPC протоколы
- | | └─ news.proto
- | └─ main.py # Точка входа
- | └─ init_data.py # Начальные данные
- └─ templates/ # HTML шаблоны
- | └─ base.html
- | └─ index.html
- | └─ login.html

- | └─ register.html
- | └─ profile.html
- | └─ news_detail.html
- | └─ create.html
- | └─ edit.html
- └─ static/ # Статические файлы
- └─ requirements.txt # Python зависимости
- └─ Dockerfile # Docker образ
- └─ docker-compose.yml # Оркестрация

6. ФУНКЦИОНАЛЬНЫЕ ВОЗМОЖНОСТИ

6.1 Для анонимных пользователей

- Просмотр списка новостей
- Чтение полных статей
- Поиск по новостям
- Регистрация нового аккаунта

6.2 Для авторизованных (reader)

- Всё выше +
- Профиль пользователя
- История прочитанных новостей
- Сохранение предпочтений

6.3 Для авторов (author)

- Всё выше +
- Создание новых статей
- Редактирование своих статей
- Удаление своих статей
- Просмотр своих публикаций в профиле

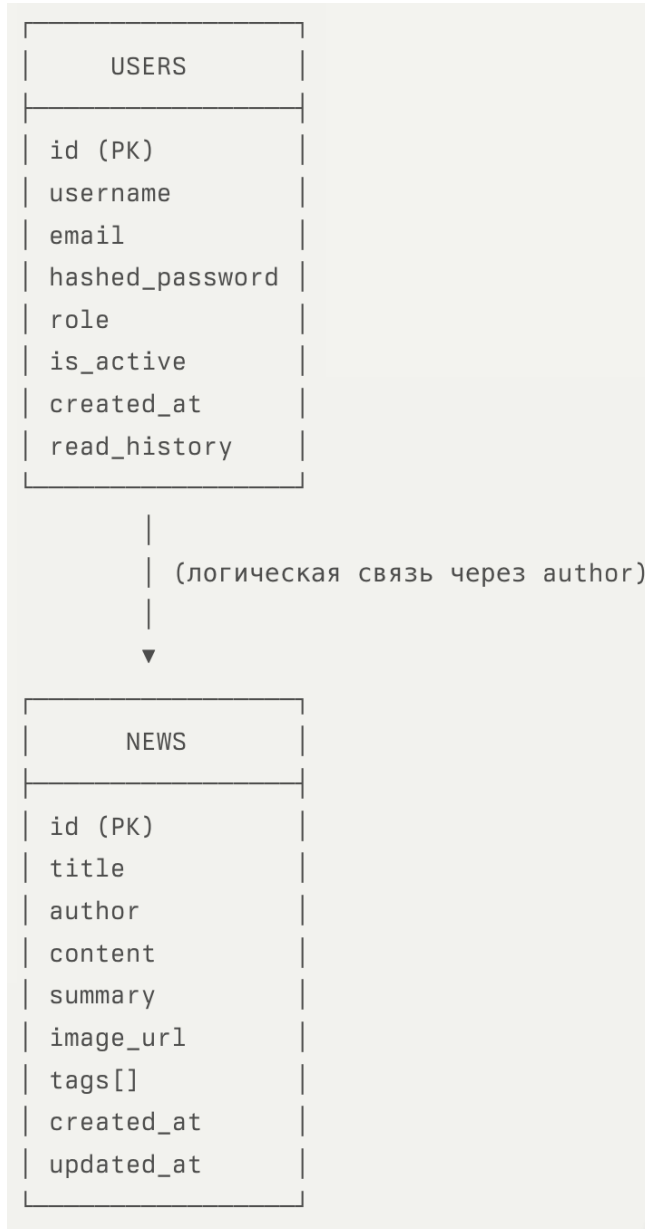
6.4 Для администраторов (admin)

- Полный доступ ко всему контенту
- Управление любыми новостями

- Редактирование/удаление любых статей
- Изменение авторства

7. БАЗА ДАННЫХ

7.1 ER-диаграмма



7.2 Индексы

```
CREATE INDEX idx_users_username ON users(username);
```

```
CREATE INDEX idx_users_email ON users(email);
```

```
CREATE INDEX idx_news_author ON news(author);
```

```
CREATE INDEX idx_news_created_at ON news(created_at DESC);
```

7.3 Запросы

Поиск новостей:

```
db.query(News).filter(  
    (News.title.ilike(f"%{query}%")) |  
    (News.content.ilike(f"%{query}%"))  
).all()
```

Новости автора:

```
db.query(News).filter(News.author ==  
    username).order_by(News.created_at.desc()).all()
```

8. API ДОКУМЕНТАЦИЯ

8.1 REST API

FastAPI автоматически генерирует OpenAPI документацию:

Swagger UI: <http://localhost:8000/docs>

ReDoc: <http://localhost:8000/redoc>

8.2 Основные endpoints

Аутентификация

POST /auth/register - Регистрация

POST /auth/login - Авторизация

GET /profile - Профиль пользователя

Новости (публичные)

GET / - Главная страница со списком новостей

GET /news/{id} - Детальная страница новости

GET /search?q=... - Поиск по новостям

Новости (защищённые)

GET /create - Форма создания новости

POST /create - Создание новости (author/admin)

GET /edit/{id} - Форма редактирования

POST /edit/{id} - Обновление новости (author/admin)

POST /delete/{id} - Удаление новости (admin)

API endpoints (JSON)

GET /api/news - Список всех новостей

GET /api/news/{id} - Получить новость по ID

POST /api/news - Создать новость

PUT /api/news/{id} - Обновить новость

DELETE /api/news/{id} - Удалить новость

GET /api/news/search?q= - Поиск новостей

8.3 gRPC API

NewsService.GetAllNews()

NewsService.GetNewsById(id)

NewsService.CreateNews(data)

NewsService.UpdateNews(id, data)

NewsService.DeleteNews(id)

NewsService.SearchNews(query)

9. РАЗВЁРТЫВАНИЕ

9.1 Требования

- Docker 24.0+
- Docker Compose 2.0+
- 2 GB RAM
- 5 GB свободного места

9.2 Запуск

Клонировать репозиторий

```
git clone https://github.com/IAlehkaI/LABA3.git
```

```
cd LABA3
```

Запустить все сервисы

```
docker compose up --build
```

Приложение доступно на:

- Web UI: <http://localhost:8000>

- API Docs: <http://localhost:8000/docs>

- gRPC: localhost:50051

- MinIO: <http://localhost:9001>

- PostgreSQL: localhost:5432

9.3 Тестовые аккаунты

Роль	Логин	Пароль
Администратор	admin	admin
Автор	author	author123
Читатель	reader	reader123

9.4 Остановка

`docker compose down -v` # С удалением данных

`docker compose down` # Без удаления данных

№	Требование	Статус	Реализация
1	Frontend приложение (React/Vue/Angular)	Частично	Используется SSR + JS вместо SPA
2.1	PostgreSQL + ORM + Repository	Полностью	SQLAlchemy 2.0, CRUD слой
2.2	Docker образ	Полностью	Dockerfile готов
2.3	Docker Compose	Полностью	4 сервиса
2.4	Аутентификация API	Полностью	JWT токены
3.1	Ролевая модель	Полностью	4 роли
3.2	gRPC сервис	Полностью	Порт 50051
3.3	S3 хранилище	Полностью	MinIO
3.4	Redis кэширование	Полностью	Распределённый кэш
3.5	Docker Compose для всех	Полностью	docker-compose.yml

10.2 Технические достижения

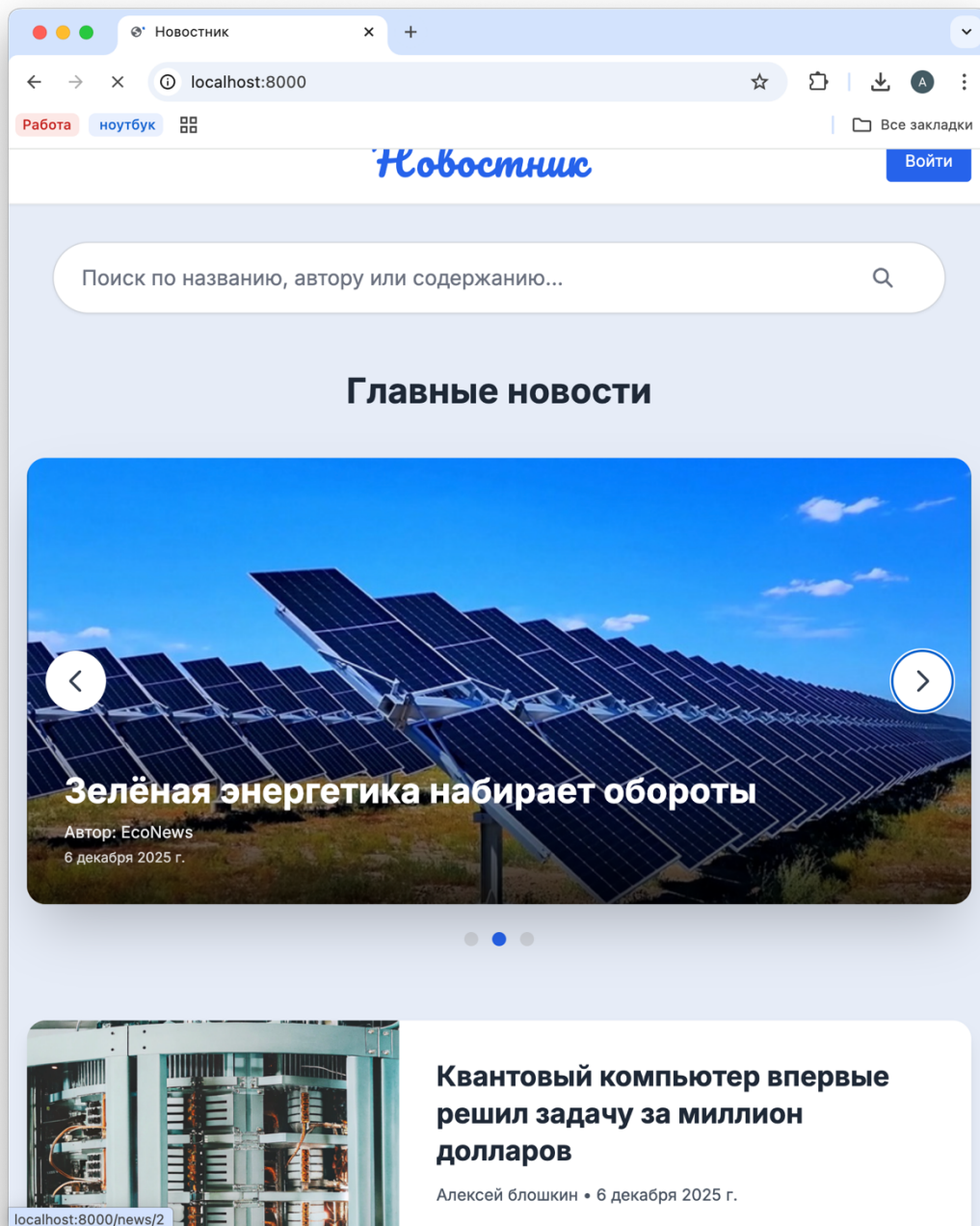
- Микросервисная архитектура
- REST + gRPC одновременно
- Полноценная JWT аутентификация
- 4-уровневая ролевая модель
- S3-хранилище для медиа
- Redis кэширование
- PostgreSQL с индексами
- ORM (SQLAlchemy 2.0)
- Repository паттерн
- Dependency Injection
- Docker контейнеризация
- Автоматическая документация API

10.3 Дополнительные возможности

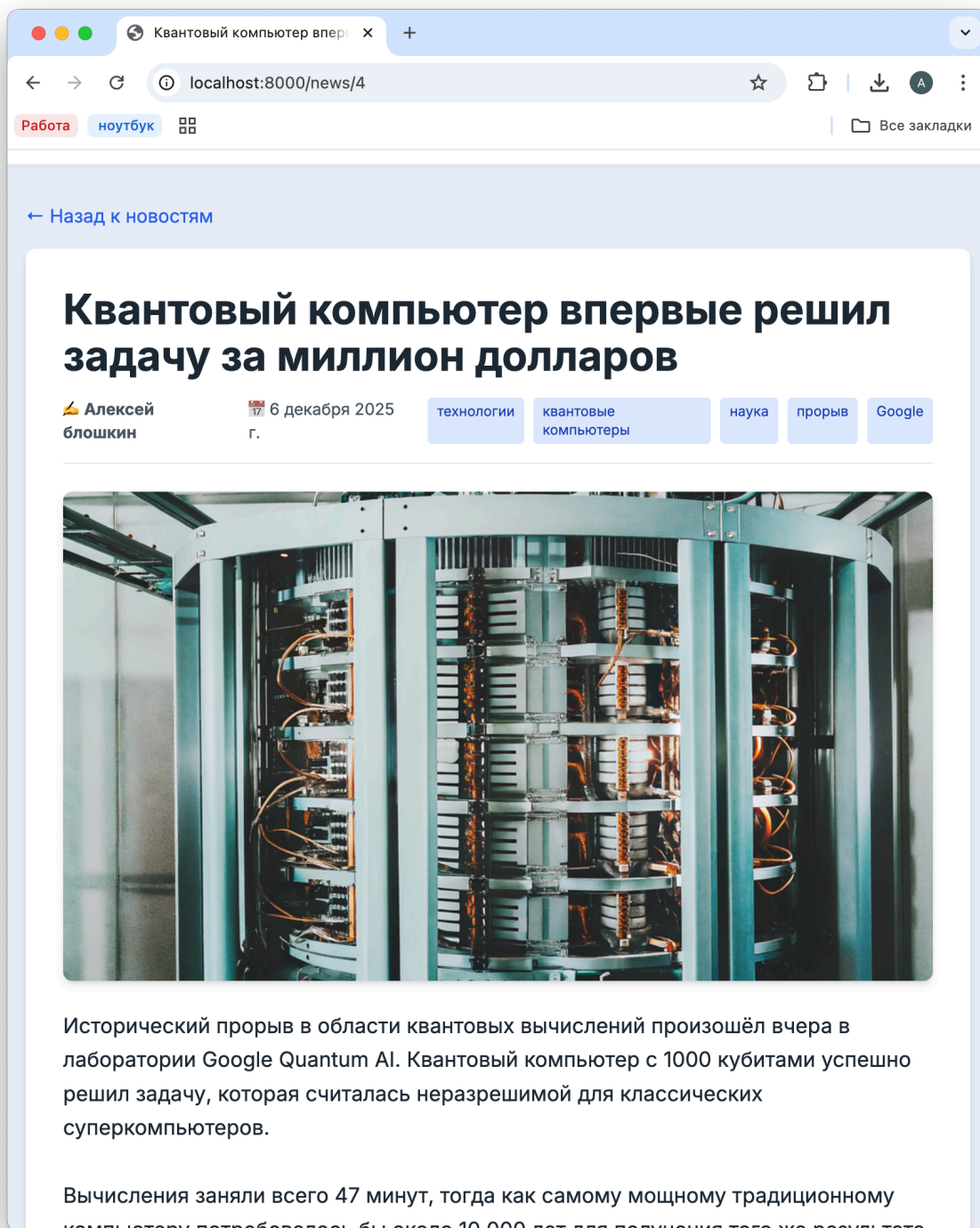
- Регистрация новых пользователей
- Профили с историей чтения
- Поиск по новостям
- Загрузка изображений
- Система тегов
- Автоматическое обновление UI
- Адаптивный дизайн
- Валидация данных (Pydantic)
- Обработка ошибок
- Логирование

Скриншоты:

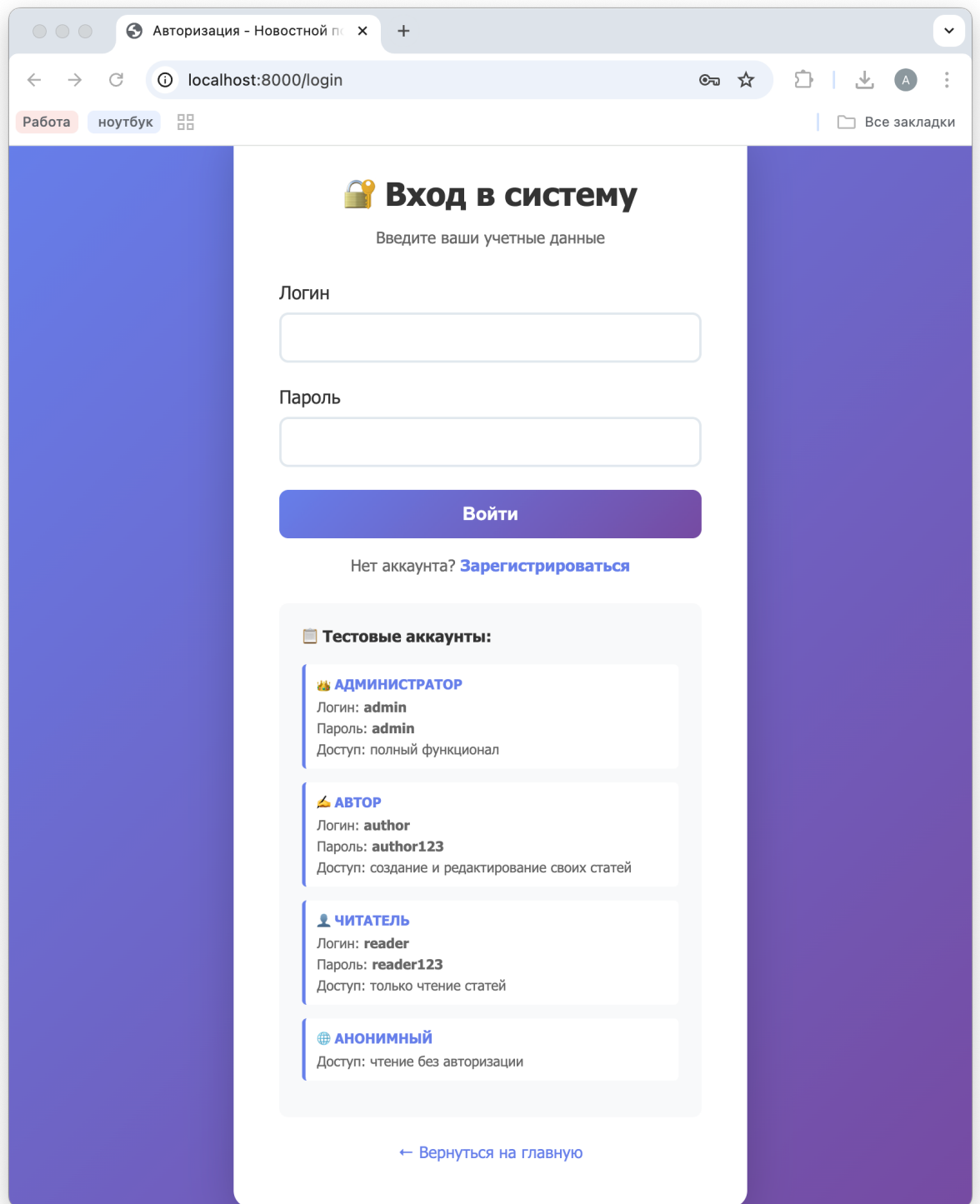
1 Главная страница-анонимный пользователь



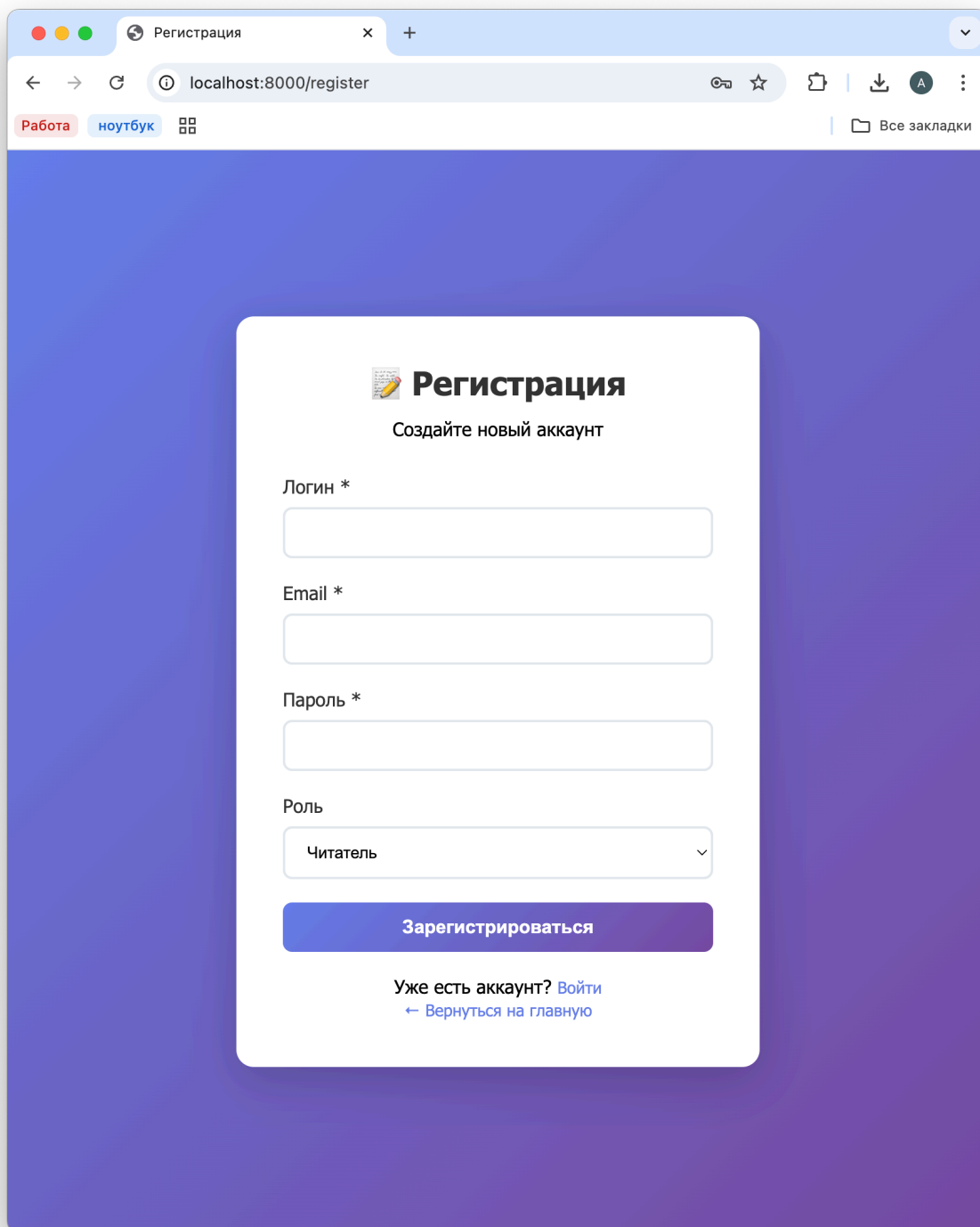
страница статьи - анонимный пользователь



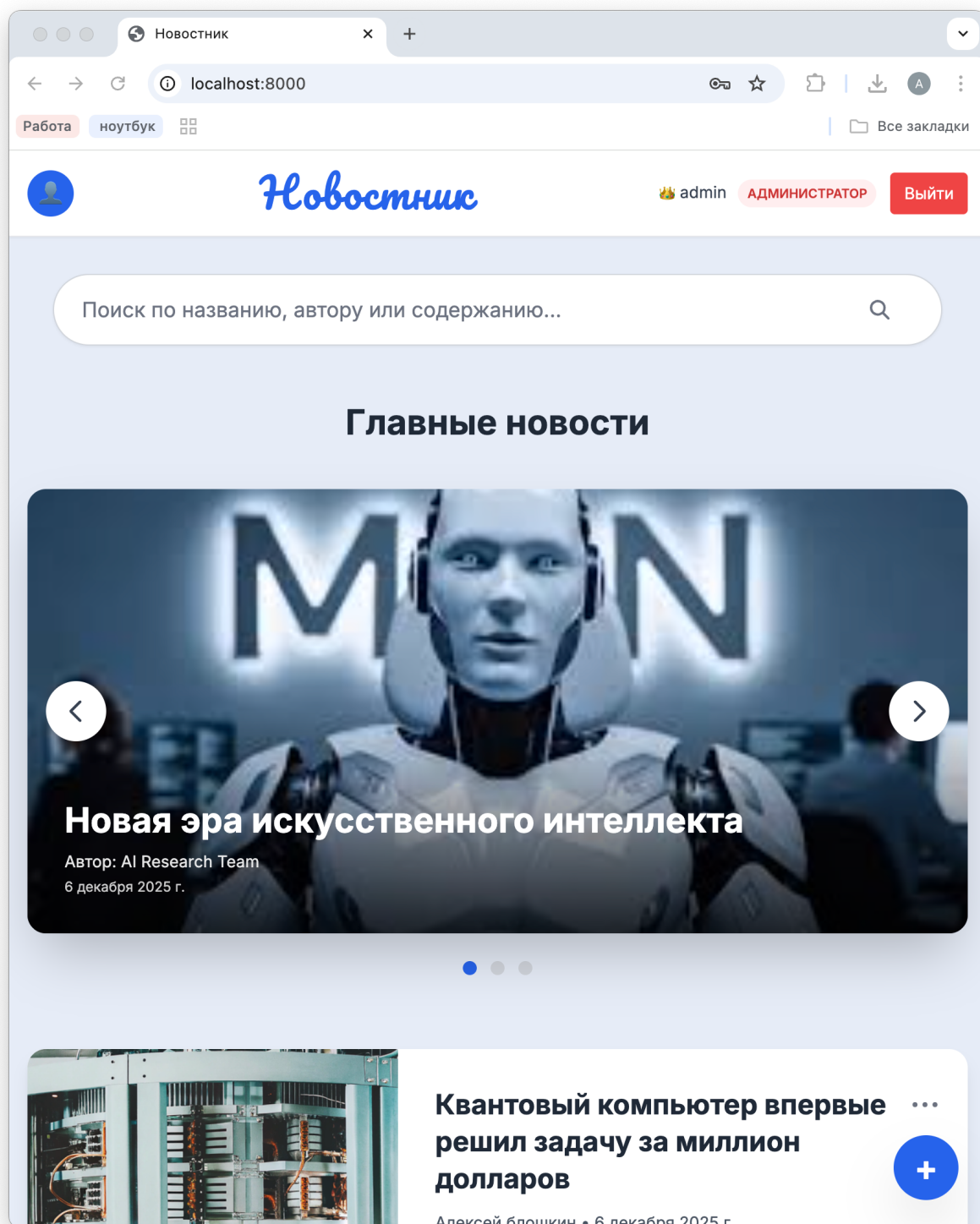
Страница авторизации



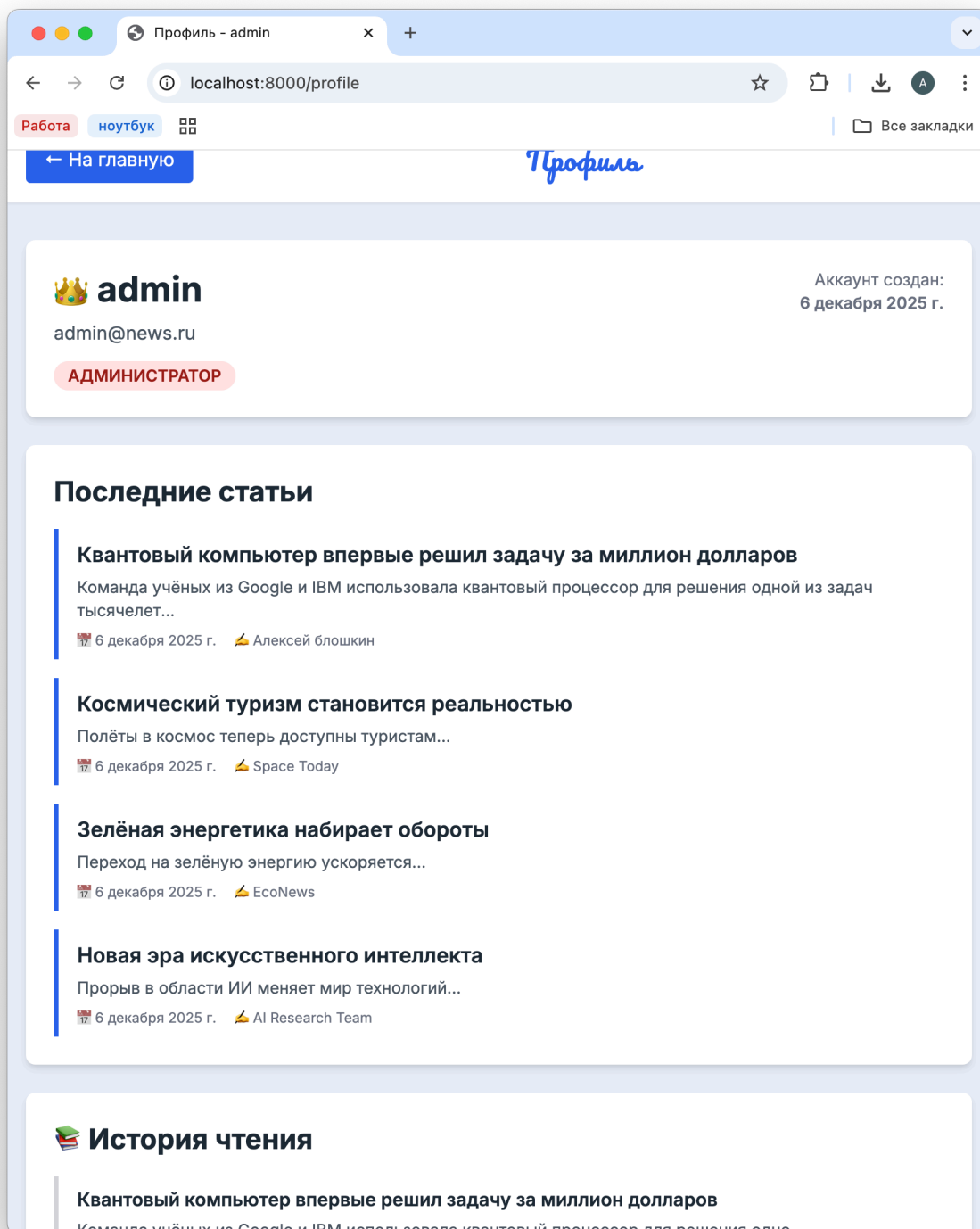
Страница регистрации:



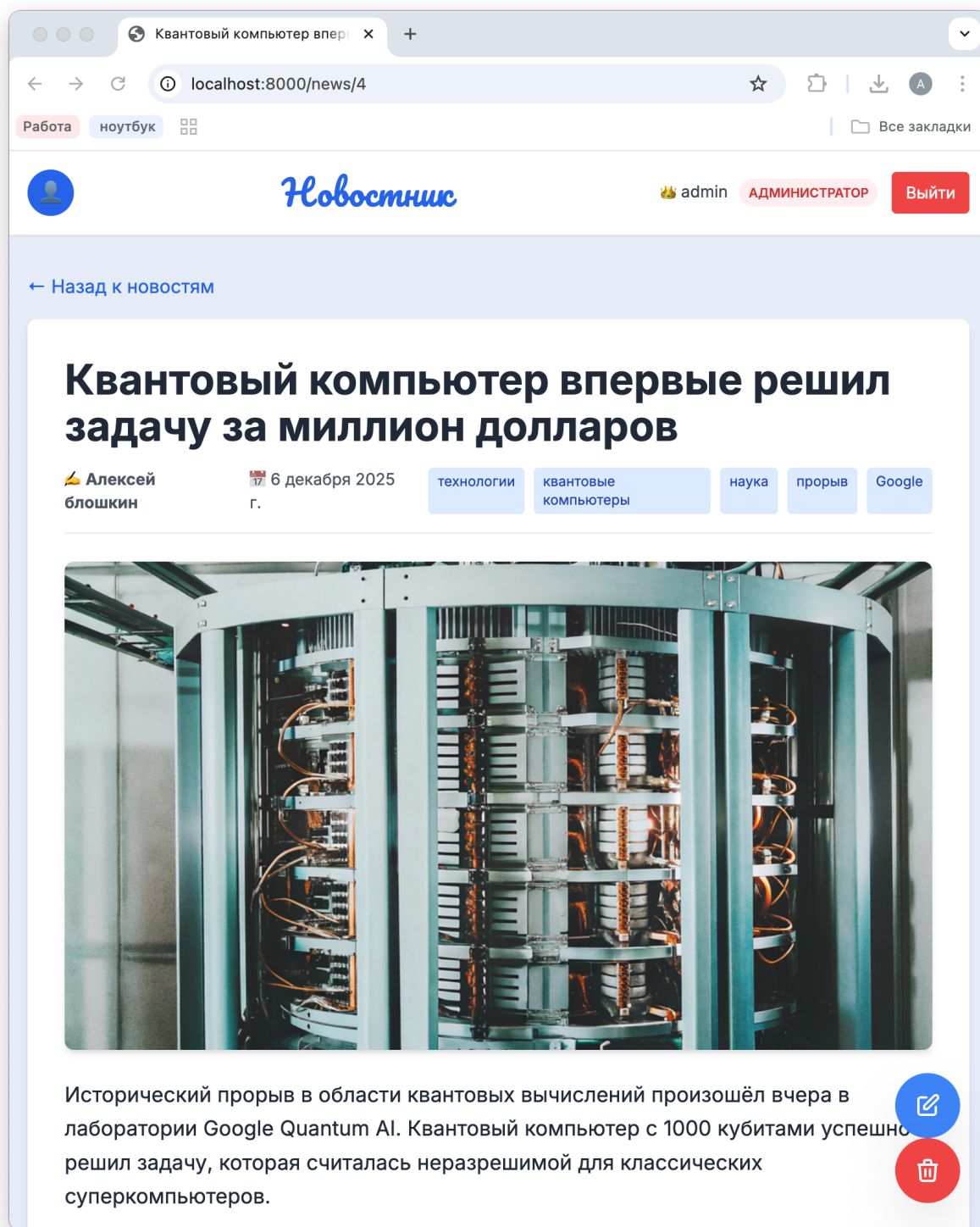
Главная страница – админ



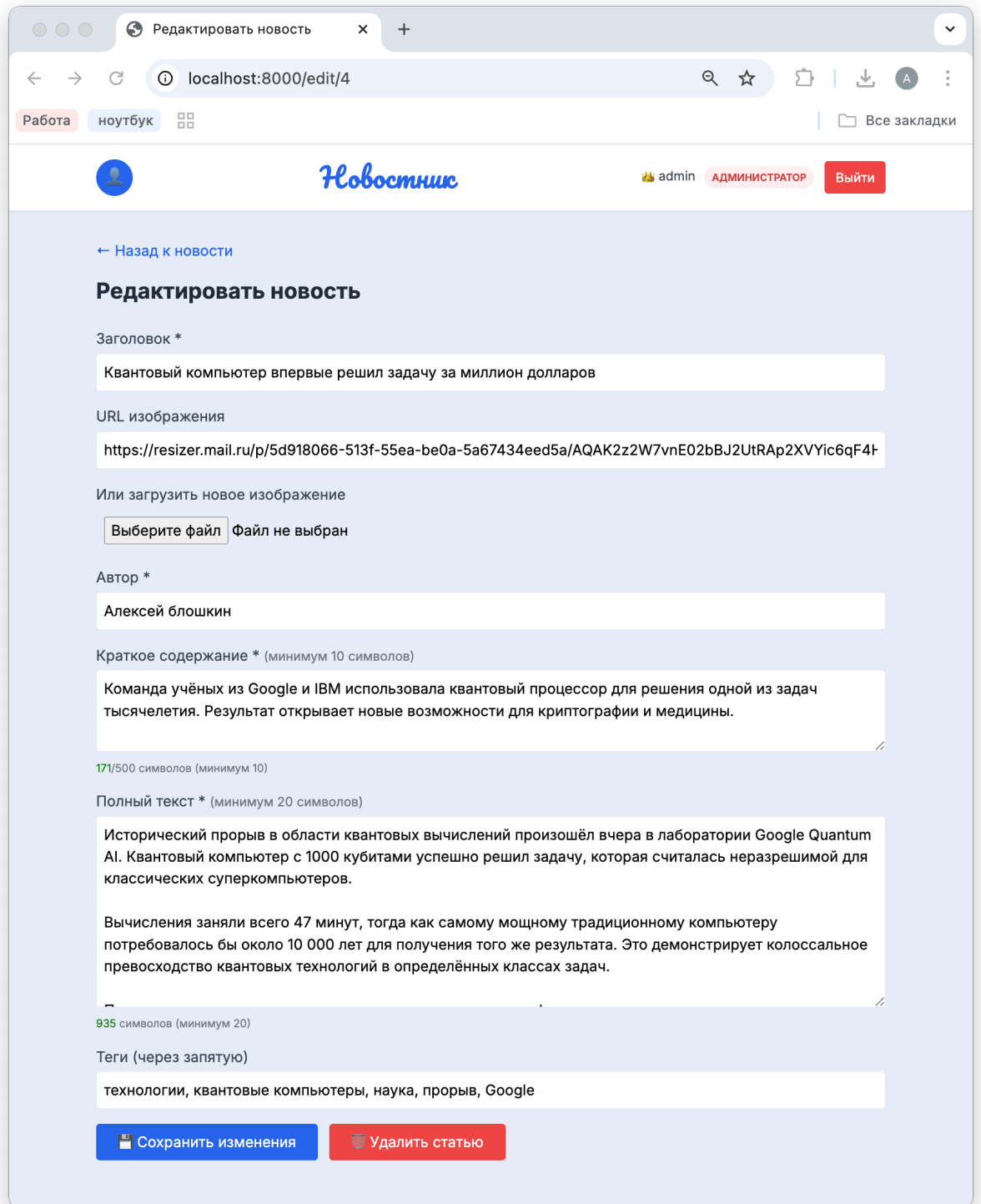
Страница профиля-админ



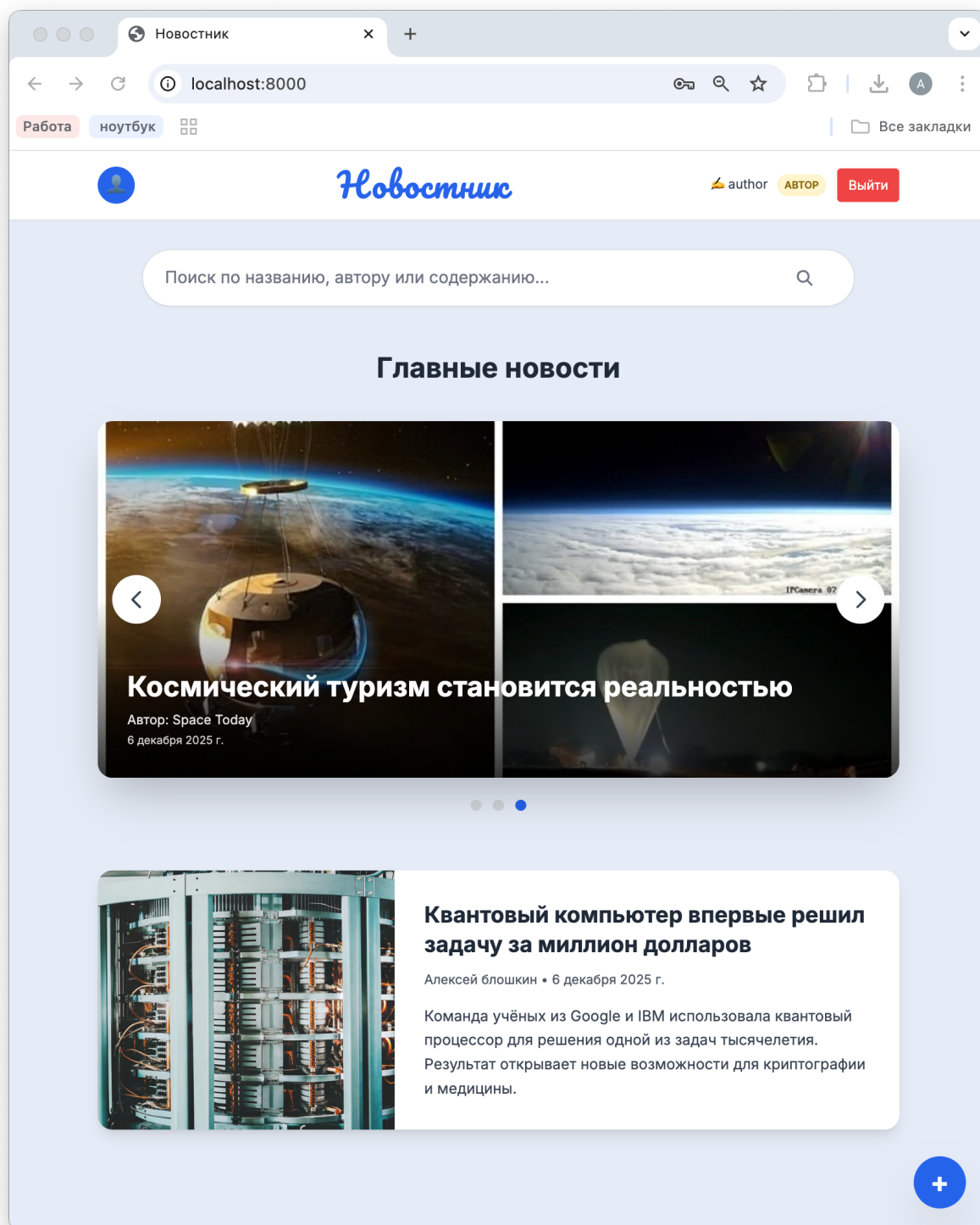
Страница статьи- админ



Страница редактирования статьи – админ



Главная страница – автор



Создание новости – автор

Добавить новость

localhost:8000/create

Работа

ноутбук

Все закладки



Новостник

 author

АВТОР

Выйти

[← Назад](#)

Добавить новость

Заголовок *

Статья 1

URL изображения

https://example.com/image.jpg

Или загрузить изображение

Выберите файл

Файл не выбран

Автор *

author

Для авторов имя заполняется автоматически

Краткое содержание * (минимум 10 символов)

Краткое содержание

18/500 символов (минимум 10)

Полный текст * (минимум 20 символов)

полный текст статьи!

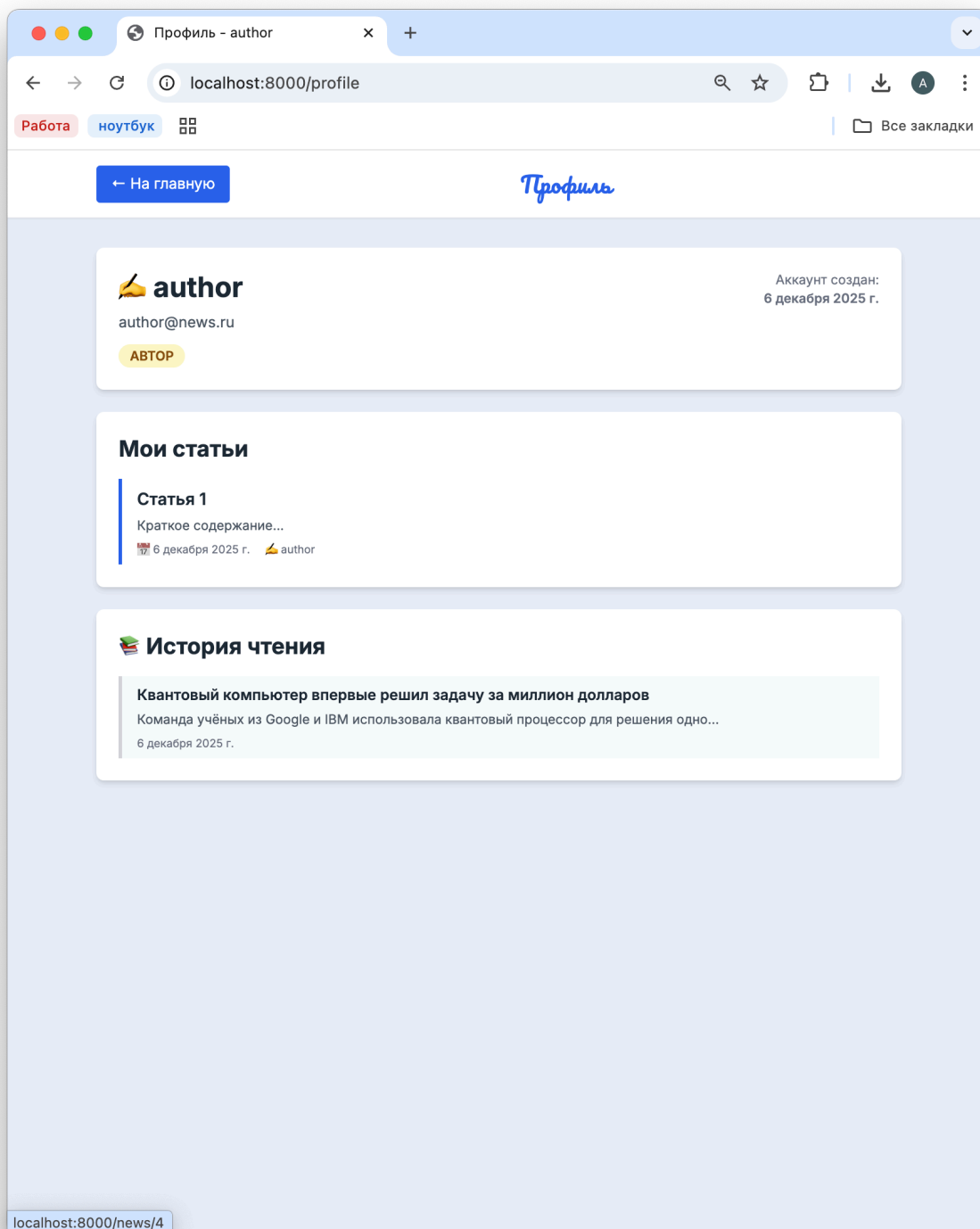
20 символов (минимум 20)

Теги (через запятую)

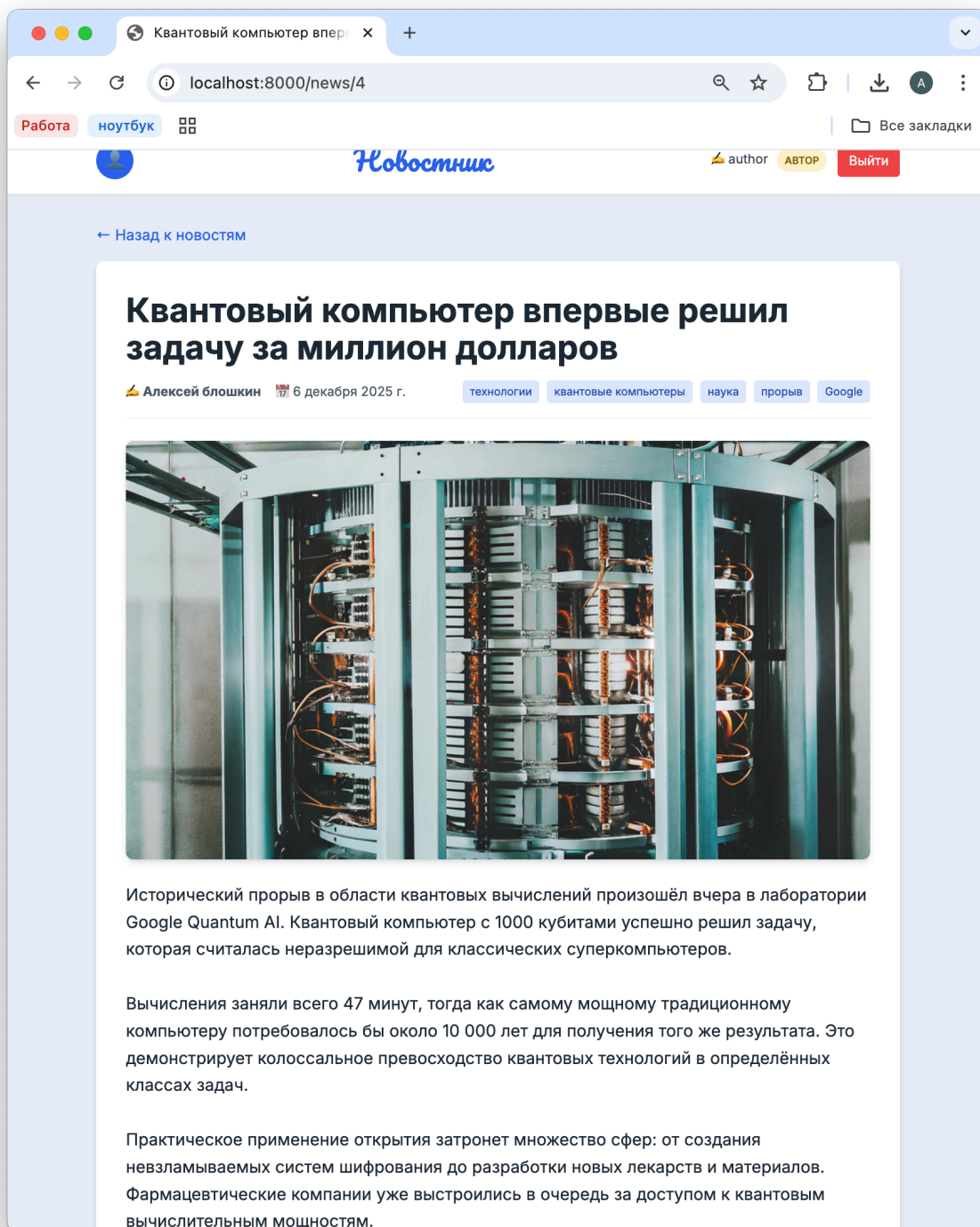
проба

Добавить новость

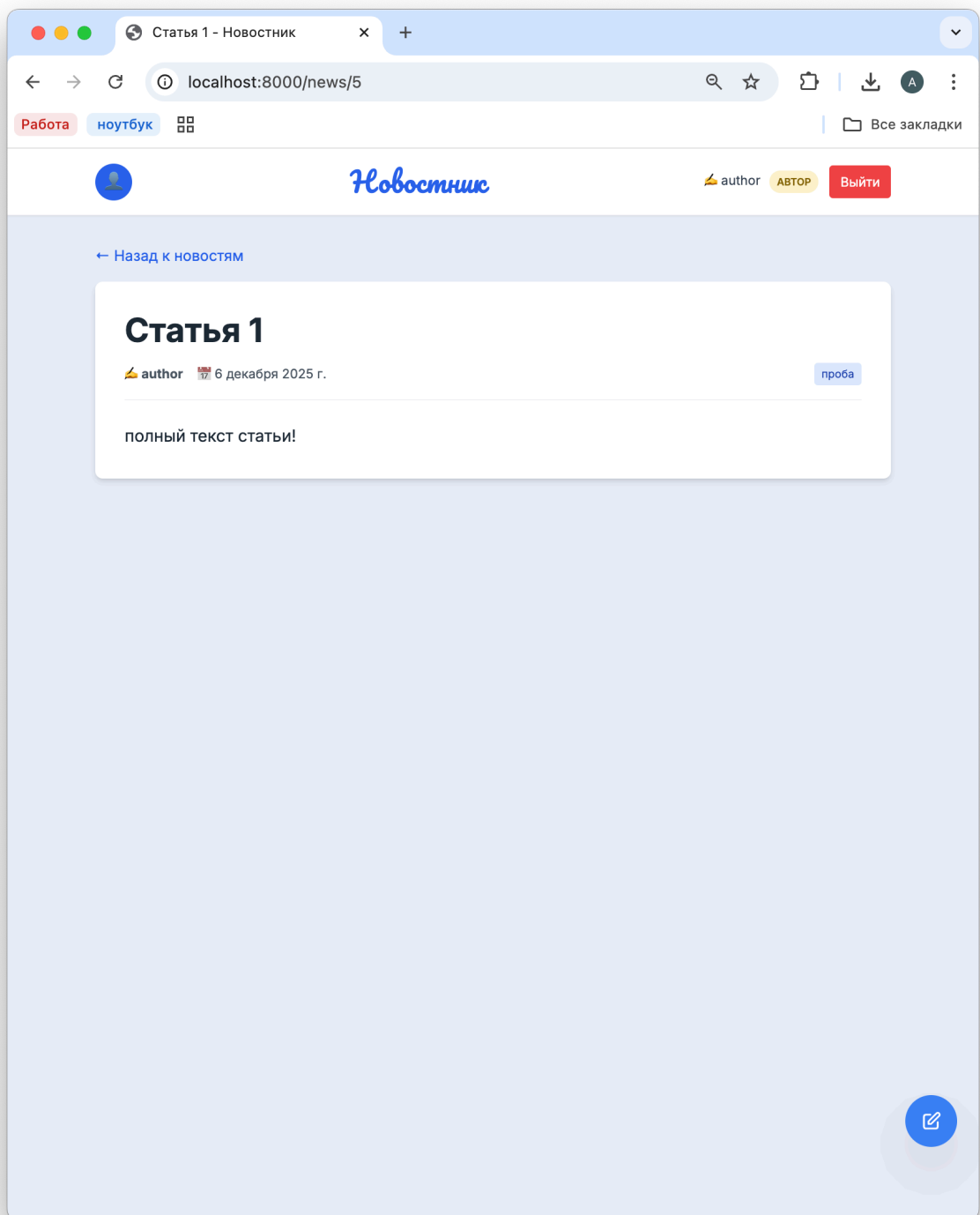
Страница профиля – автор



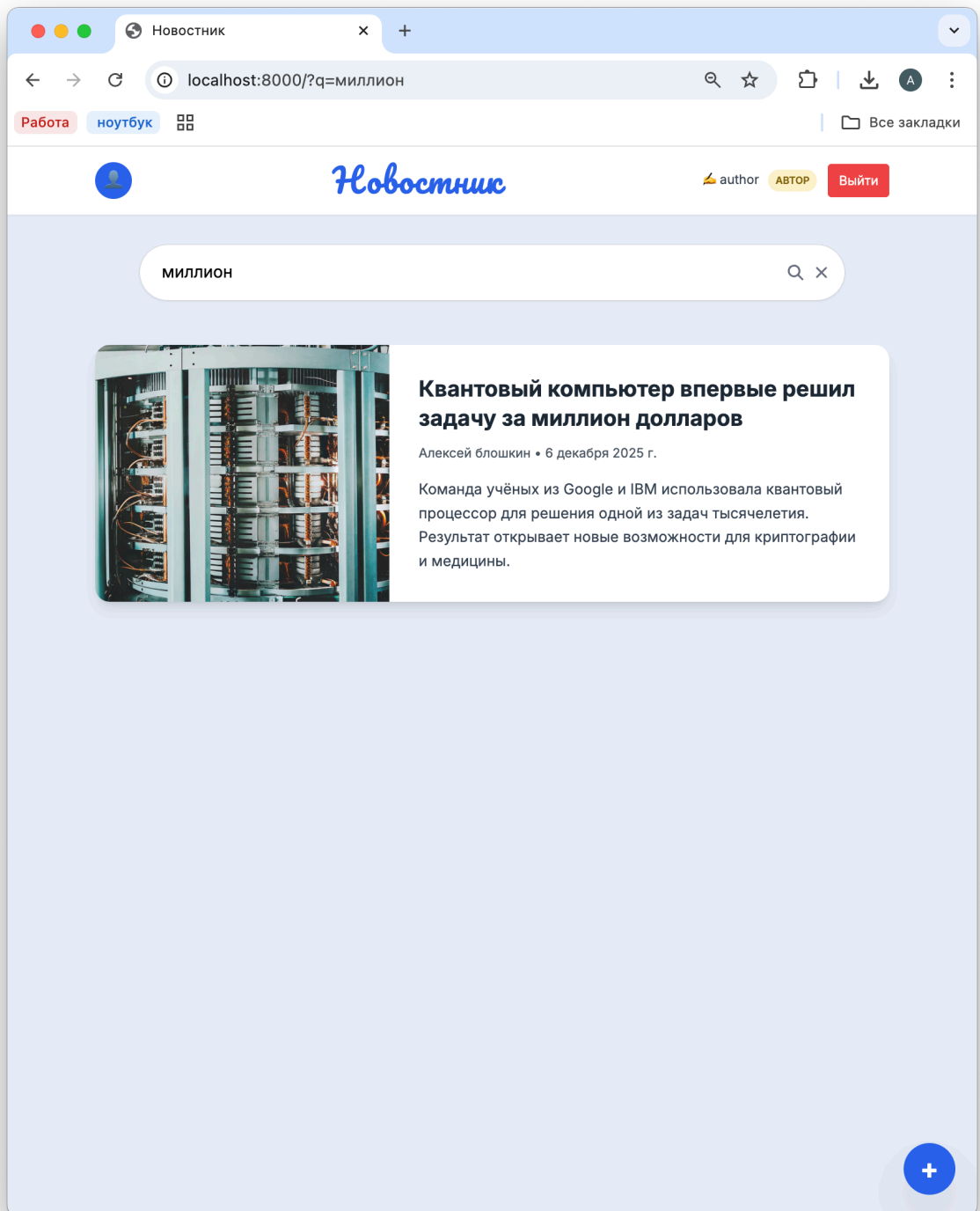
Страница статьи чужого автора – автор



Страница своей статьи – автор



Поиск



Swagger ui

